



UNIVERSIDAD CATÓLICA DE SALTA
Licenciatura en Ciencia de Datos

Exámen Final Integrador

Programación I

Alumno: Facundo Luna

Profesor: Ing. Mario Martínez

Fecha: 10 de junio de 2026

Índice

1. Enunciado	3
1.1. Funcionalidades principales	3
1.2. Conceptos del curso aplicados	3
2. Arquitectura del sistema	4
2.1. Estructura de paquetes	4
2.2. Descripción de las capas	4
2.3. Decisiones de diseño destacadas	5
2.4. Diagrama UML de clases	6
3. Código fuente	7
3.1. Capa de modelos	7
3.1.1. Clase abstracta <code>Cuenta</code>	7
3.1.2. Clase <code>CajaDeAhorro</code>	9
3.1.3. Clase <code>CuentaCorriente</code>	10
3.1.4. Clase <code>Cliente</code>	11
3.1.5. Clase <code>Movimiento</code>	13
3.1.6. Clase <code>Banco</code>	14
3.2. Capa de servicio	16
3.2.1. Clase <code>BancoService</code>	16
3.3. Capa de utilidades	20
3.3.1. Clase <code>Consola</code>	20
3.4. Menú principal	22
3.4.1. Clase <code>MenuPrincipal</code>	22
3.4.2. Clase <code>App</code> — punto de entrada	27
4. Capturas de ejecución	28
4.1. Inicio del sistema y datos de prueba	28
4.2. Gestión de clientes	29
4.3. Apertura de cuenta y consulta	30

4.4. Transferencia entre cuentas	30
4.5. Historial de movimientos y saldo	31

1. Enunciado

Se desarrolla un **Sistema de Gestión Bancaria** por consola que permite administrar clientes, cuentas bancarias y movimientos financieros. El sistema contempla dos tipos de cuentas: *Caja de Ahorro* y *Cuenta Corriente*, ambas derivadas de una clase común.

1.1. Funcionalidades principales

El sistema permite realizar las siguientes operaciones:

1. **Alta de clientes** — registro de nombre, DNI y datos de contacto.
2. **Apertura de cuentas** — Caja de Ahorro o Cuenta Corriente asociada a un cliente.
3. **Depósito y extracción** — con validación de saldo disponible y límite de descubierto.
4. **Transferencias entre cuentas** — con registro automático de movimientos.
5. **Consulta de saldo e historial** — listado de movimientos ordenados por fecha.
6. **Búsqueda de clientes y cuentas** — por DNI o número de cuenta.
7. **Listar clientes con mayor saldo** — usando `Stream` y comparadores funcionales.

1.2. Conceptos del curso aplicados

Concepto	Aplicación en el sistema
Operaciones y cálculos secuenciales	Depósito, extracción, transferencia
Ingreso de datos por consola	Menú interactivo con <code>Scanner</code>
Estructuras de selección	<code>switch</code> en menú, <code>if</code> en validaciones
Bucles	<code>while</code> y <code>for-each</code> en listados
Creación de clases	<code>Cliente</code> , <code>Cuenta</code> , <code>Movimiento</code> , ...
Creación de objetos	Instanciación desde menú y servicio
Arreglos / colecciones	<code>List<></code> para cuentas, movimientos, clientes
Herencia	<code>CajaDeAhorro</code> y <code>CuentaCorriente</code> extends <code>Cuenta</code>

2. Arquitectura del sistema

2.1. Estructura de paquetes

El proyecto adopta una **arquitectura en capas** inspirada en el patrón *Model-Service-View*, adaptada al contexto de consola. Cada paquete tiene una responsabilidad única y acotada:

src/		
menu/		
MenuPrincipal.java	—	Punto de entrada e interacción con el usuario
modelos/		
Banco.java	—	Entidad raíz del sistema
Cliente.java	—	Datos del titular
Cuenta.java	—	Clase abstracta base
CajaDeAhorro.java	—	Especialización de Cuenta
CuentaCorriente.java	—	Especialización con descubierto
Movimiento.java	—	Registro de operaciones
servicio/		
BancoService.java	—	Lógica de negocio centralizada
utilidades/		
Consola.java	—	Lectura segura de entradas del usuario
App.java	—	Arranque de la aplicación

2.2. Descripción de las capas

modelos/ Contiene las clases de dominio puro: definen atributos, constructores, getters/setters y, en el caso de **Cuenta**, métodos abstractos que las subclases implementan (**depositar**, **extraer**). No contienen lógica de negocio ni acceso a la UI.

servicio/

BancoService encapsula todas las reglas del negocio: alta de clientes y cuentas, transferencias, búsquedas y reportes. Es la única capa que manipula las colecciones internas del **Banco**. El menú solo llama métodos de servicio; nunca accede al modelo directamente.

utilidades/

Consola centraliza la lectura de datos del usuario, maneja excepciones de entrada (**NumberFormatException**) y expone métodos reutilizables (**leerEntero**,

`leerTexto`, `leerDouble`). Al estar en un paquete propio puede usarse desde cualquier capa sin generar dependencias circulares.

menu/ `MenuPrincipal` es la capa de presentación: muestra opciones, captura la selección y delega en `BancoService`. Toda la interacción con el usuario ocurre aquí.

2.3. Decisiones de diseño destacadas

- **`Optional<T>` — manejo seguro de nulos**

Los métodos de búsqueda (`buscarClientePorDni`, `buscarCuentaPorNumero`) retornan `Optional` en lugar de `null`. Esto obliga al llamador a manejar el caso “no encontrado” de forma explícita, eliminando los `NullPointerException`.

- **`Stream` y programación funcional**

Se usan `stream().filter()`, `.map()`, `.sorted()` y `.collect()` para recorrer y transformar colecciones sin bucles imperativos. Por ejemplo, listar los clientes ordenados por saldo total se resuelve en una sola expresión en lugar de un bucle con comparadores manuales.

- **Herencia e `instanceof`**

`CajaDeAhorro` y `CuentaCorriente` extienden la clase abstracta `Cuenta`. El polimorfismo permite operar sobre cualquier cuenta sin conocer su tipo concreto; `instanceof` se emplea únicamente donde la lógica difiere (por ejemplo, el descubierto).

- **Separación total UI / lógica**

El servicio nunca imprime por consola; el menú nunca accede al estado interno del banco. Esta separación facilita agregar una interfaz gráfica en el futuro sin tocar la lógica de negocio.

2.4. Diagrama UML de clases

La figura 1 muestra las relaciones entre las clases del sistema.

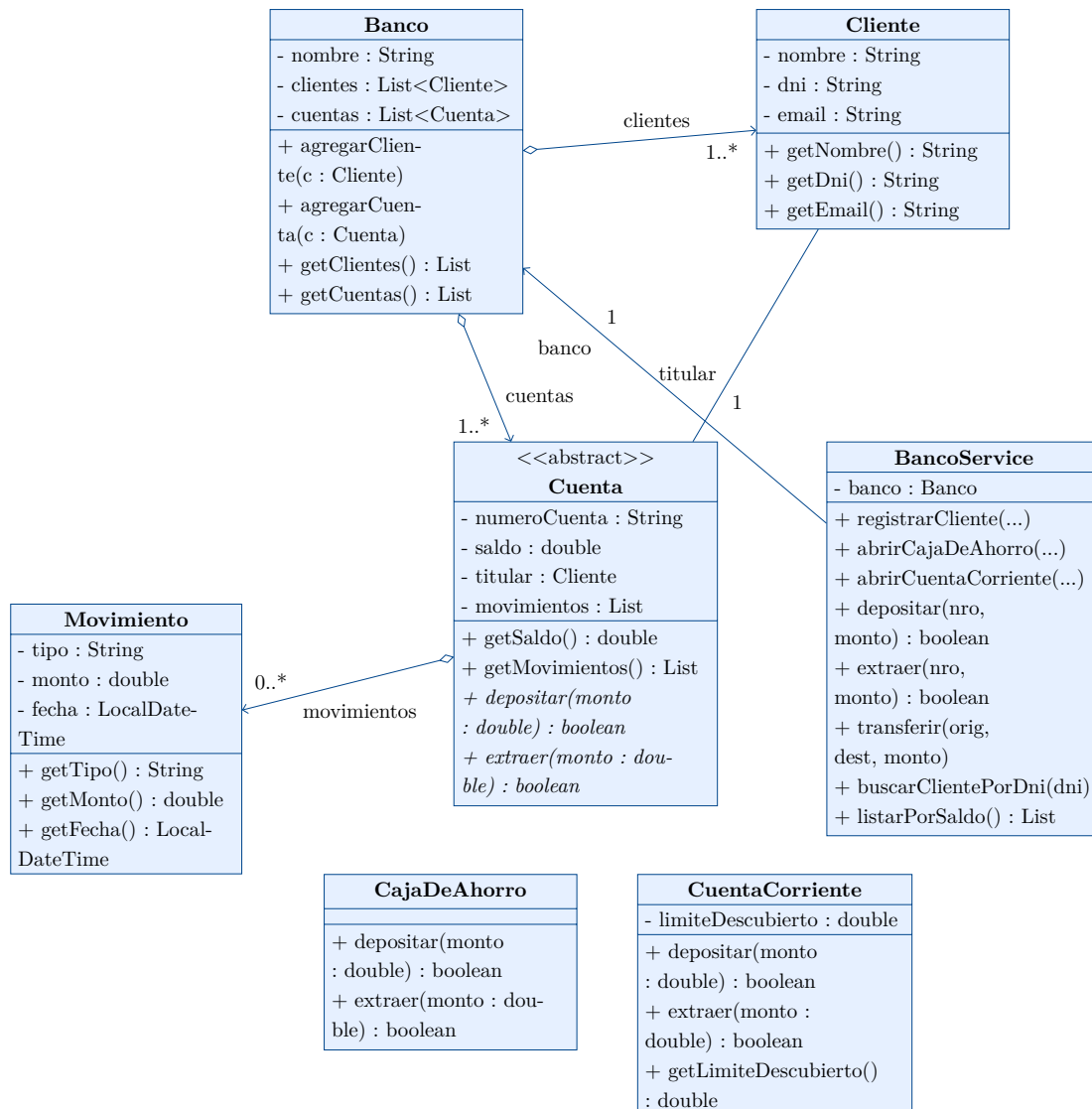


Figura 1: Diagrama UML de clases — Sistema Bancario

3. Código fuente

El código completo está disponible en el repositorio:

<https://github.com/Facug1/data-science-coursework/tree/main/primer-anio/programacion-1/parcial-integrador/sistema-bancario/src>

A continuación se presenta el código de cada clase, organizado por capa.

3.1. Capa de modelos

3.1.1. Clase abstracta Cuenta

```
package modelos;

import java.util.ArrayList;
import java.util.List;

public abstract class Cuenta {
    private final String numeroCuenta;
    private double saldo;
    private final Cliente titular;
    private final List<Movimiento> historialMovimientos;

    public Cuenta(String numeroCuenta, double saldoInicial, Cliente titular) {
        this.numeroCuenta = numeroCuenta;
        this.saldo = saldoInicial;
        this.titular = titular;
        this.historialMovimientos = new ArrayList<>();

        // Registro del movimiento inicial de apertura
        if (saldoInicial > 0) {
            registrarMovimiento(Movimiento.Tipo.DEPOSITO, saldoInicial, "Apertura de
cuenta");
        }
    }

    public boolean depositar(double monto) {
        if (monto <= 0) {
            return false;
        }
        this.saldo += monto;
        registrarMovimiento(Movimiento.Tipo.DEPOSITO, monto, "Depósito en efectivo");
        return true;
    }

    public boolean extraer(double monto) {
        if (monto <= 0 || monto > this.saldo) {
```



```
        return false;
    }
    this.saldo -= monto;
    registrarMovimiento(Movimiento.Tipo.EXTRACCION, monto, "Extracción en
efectivo");
    return true;
}

// Registra un movimiento en el historial de la cuenta
protected void registrarMovimiento(Movimiento.Tipo tipo, double monto, String
descripcion) {
    Movimiento movimiento = new Movimiento(tipo, monto, this.saldo, descripcion);
    this.historialMovimientos.add(movimiento);
}

// Acá usaremos Pliformismo tambien, cada subclase implementa su propia
descripción
public abstract String getTipoCuenta();

public String getNumeroCuenta() {
    return numeroCuenta;
}

public double getSaldo() {
    return saldo;
}

public void setSaldo(double saldo) {
    this.saldo = saldo;
}

public Cliente getTitular() {
    return titular;
}

public List<Movimiento> getHistorialMovimientos() {
    return historialMovimientos;
}

@Override
public String toString() {
    return String.format(" Cuenta %-10s | Tipo: %-18s | Titular: %-20s | Saldo:
$%.2f",
        numeroCuenta,
        getTipoCuenta(),
        titular.getNombreCompleto(),
        saldo);
}
}
```

Clase : modelos/Cuenta.java — clase base abstracta

3.1.2. Clase CajaDeAhorro

```
package modelos;

public class CajaDeAhorro extends Cuenta {
    private double tasaInteres;

    public CajaDeAhorro(String numeroCuenta, double saldoInicial, Cliente titular) {
        super(numeroCuenta, saldoInicial, titular);
        this.tasaInteres = 0.05 // Tasa anual por defecto
    };

    // Calcula y acredita los intereses correspondientes al saldo actual
    public double acreditarIntereses() {
        double interes = getSaldo() * tasaInteres;
        setSaldo(getSaldo() + interes);
        registrarMovimiento(Movimiento.Tipo.DEPOSITO, interes,
            String.format("Acreditación de intereses (0.1f%% anual)",
                tasaInteres * 100));
        return interes;
    }

    @Override
    public String getTipoCuenta() {
        return "Caja de Ahorro";
    }

    public double getTasaInteres() {
        return tasaInteres;
    }

    public void setTasaInteres(double tasaInteres) {
        if (tasaInteres >= 0 && tasaInteres <= 1) {
            this.tasaInteres = tasaInteres;
        }
    }

    @Override
    public String toString() {
        return super.toString() + String.format(" | Tasa: 0.1f%%", tasaInteres *
            100);
    }
}
```

Clase : modelos/CajaDeAhorro.java — herencia de Cuenta

3.1.3. Clase CuentaCorriente

```
package modelos;

public class CuentaCorriente extends Cuenta {
    // El saldo puede bajar hasta: -limiteDescubierto
    private double limiteDescubierto;

    public CuentaCorriente(String numeroCuenta, double saldoInicial, Cliente titular,
        double limiteDescubierto) {
        super(numeroCuenta, saldoInicial, titular);
        this.limiteDescubierto = limiteDescubierto;
    }

    // Acá empleamos polimorfismo al sobrescribir extraer()
    @Override
    public boolean extraer(double monto) {
        if (monto <= 0) {
            return false;
        }

        double saldoDisponible = getSaldo() + limiteDescubierto;

        if (monto > saldoDisponible) {
            return false; // supera el límite de descubierto
        }

        setSaldo(getSaldo() - monto);

        String descripcion = getSaldo() < 0
            ? String.format("Extracción en descubierto (límite: $%.2f)",
                limiteDescubierto)
            : "Extracción en efectivo";

        registrarMovimiento(Movimiento.Tipo.EXTRACCION, monto, descripcion);
        return true;
    }

    // Verifica si la cuenta se encuentra actualmente en descubierto
    public boolean estaEnDescubierto() {
        return getSaldo() < 0;
    }

    // Calcula el monto disponible para extraer (saldo + descubierto)
    public double getMontoDisponible() {
        return getSaldo() + limiteDescubierto;
    }

    @Override
    public String getTipoCuenta() {
        return "Cuenta Corriente";
    }
}
```

```
}

public double getLimiteDescubierto() {
    return limiteDescubierto;
}

public void setLimiteDescubierto(double limiteDescubierto) {
    if (limiteDescubierto >= 0) {
        this.limiteDescubierto = limiteDescubierto;
    }
}

@Override
public String toString() {
    String estado = estaEnDescubierto() ? " [EN DESCUBIERTO]" : "";
    return super.toString()
        + String.format(" | Descubierto: $%.2f%s", limiteDescubierto, estado);
}
}
```

Clase : modelos/CuentaCorriente.java — descubierto autorizado

3.1.4. Clase Cliente

```
package modelos;

import java.util.ArrayList;
import java.util.List;

public class Cliente {
    private String nombre;
    private String apellido;
    private final String dni;
    private String email;
    private final List<Cuenta> cuentas;

    public Cliente(String nombre, String apellido, String dni, String email) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.dni = dni;
        this.email = email;
        this.cuentas = new ArrayList<>();
    }

    public void agregarCuenta(Cuenta cuenta) {
        if (cuenta != null && !cuentas.contains(cuenta)) {
            cuentas.add(cuenta);
        }
    }
}
```

```
public int cantidadCuentas() {
    return cuentas.size();
}

public String getNombreCompleto() {
    return apellido + ", " + nombre;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
}

public String getDni() {
    return dni;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public List<Cuenta> getCuentas() {
    return cuentas;
}

@Override
public String toString() {
    return String.format("  Cliente: %-25s | DNI: %-10s | Email: %-25s |
Cuentas: %d",
        getNombreCompleto(), dni, email, cuentas.size());
}
}
```

Clase : modelos/Cliente.java

3.1.5. Clase Movimiento

```
package modelos;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Movimiento {
    public enum Tipo {
        DEPOSITO("Depósito"),
        EXTRACCION("Extracción"),
        TRANSFERENCIA_ENTRADA("Transferencia entrada"),
        TRANSFERENCIA_SALIDA("Transferencia salida");

        private final String descripcion;

        Tipo(String descripcion) {
            this.descripcion = descripcion;
        }

        public String getDescripcion() {
            return descripcion;
        }
    }

    private final Tipo tipo;
    private final double monto;
    private final LocalDateTime fecha;
    private final double saldoResultante;
    private final String descripcion;
    private static final DateTimeFormatter FORMATO_FECHA =
        DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss");

    public Movimiento(Tipo tipo, double monto, double saldoResultante, String
descripcion) {
        this.tipo = tipo;
        this.monto = monto;
        this.fecha = LocalDateTime.now();
        this.saldoResultante = saldoResultante;
        this.descripcion = descripcion;
    }

    public Tipo getTipo() {
        return tipo;
    }

    public double getMonto() {
        return monto;
    }

    public LocalDateTime getFecha() {
```

```
        return fecha;
    }

    public double getSaldoResultante() {
        return saldoResultante;
    }

    public String getDescripcion() {
        return descripcion;
    }

    public static DateTimeFormatter getFormatoFecha() {
        return FORMATO_FECHA;
    }

    @Override
    public String toString() {
        return String.format(" [%s] %-25s | Monto: $%10.2f | Saldo: $%10.2f | %s",
            fecha.format(FORMATO_FECHA),
            tipo.getDescripcion(),
            monto,
            saldoResultante,
            descripcion);
    }
}
```

Clase : modelos/Movimiento.java — historial de operaciones

3.1.6. Clase Banco

```
package modelos;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class Banco {
    private final String nombre;
    private final String cbu;
    private int contadorCuentas;
    private final List<Cliente> clientes;
    private final List<Cuenta> cuentas;

    public Banco(String nombre, String cbu) {
        this.nombre = nombre;
        this.cbu = cbu;
        this.contadorCuentas = 1000;
        this.clientes = new ArrayList<>();
        this.cuentas = new ArrayList<>();
    }
}
```

```
}

// Registra un nuevo cliente si el DNI no existe
public boolean registrarCliente(Cliente cliente) {
    if (buscarClientePorDni(cliente.getDni()).isPresent()) {
        return false; // ya existe
    }
    clientes.add(cliente);
    return true;
}

// Se emplea Optional para el manejo seguro de null
public Optional<Cliente> buscarClientePorDni(String dni) {
    return clientes.stream()
        .filter(c -> c.getDni().equals(dni))
        .findFirst();
}

// Genera un número de cuenta único y secuencial
public String generarNumeroCuenta() {
    return "CBU-" + (++contadorCuentas);
}

// Registra una cuenta en el banco y la asocia al titular
public void registrarCuenta(Cuenta cuenta) {
    cuentas.add(cuenta);
    cuenta.getTitular().agregarCuenta(cuenta);
}

// Con Stream convierto en un flujo de datos y filtro la primera coincidencia
public Optional<Cuenta> buscarCuenta(String numeroCuenta) {
    return cuentas.stream()
        .filter(c -> c.getNumeroCuenta().equals(numeroCuenta))
        .findFirst();
}

public int totalClientes() {
    return clientes.size();
}

public int totalCuentas() {
    return cuentas.size();
}

public String getNombre() {
    return nombre;
}

public String getCbu() {
    return cbu;
}
```



```
public List<Cliente> getClientes() {
    return clientes;
}

public List<Cuenta> getCuentas() {
    return cuentas;
}

@Override
public String toString() {
    return String.format("Banco: %s | CBU: %s | Clientes: %d | Cuentas: %d",
        nombre, cbu, clientes.size(), cuentas.size());
}
}
```

Clase : modelos/Banco.java — entidad raiz del sistema

3.2. Capa de servicio

3.2.1. Clase BancoService

```
package servicio;

import java.util.Optional;

import modelos.Banco;
import modelos.CajaDeAhorro;
import modelos.Cliente;
import modelos.Cuenta;
import modelos.CuentaCorriente;
import utilidades.Consola;

// Esta clase es intermediaria entre el menú y el modelo
public class BancoService {
    private final Banco banco;

    public BancoService() {
        this.banco = new Banco("Banco Universitario S.A.", "0000003100073007020183");
    }

    // Registra un nuevo cliente en el sistema
    public void registrarCliente(String nombre, String apellido, String dni, String email) {
        Cliente cliente = new Cliente(nombre, apellido, dni, email);

        if (banco.registrarCliente(cliente)) {
            Consola.exito("Cliente registrado: " + cliente.getNombreCompleto());
        } else {
            Consola.error("Ya existe un cliente con DNI: " + dni);
        }
    }
}
```

```
    }  
}  
  
// Listamos todos los clientes registrados en el banco  
public void listarClientes() {  
    Consola.subtitulo("Clientes Registrados");  
    if (banco.getClientes().isEmpty()) {  
        Consola.info("No hay clientes registrados.");  
        return;  
    }  
  
    banco.getClientes().forEach(System.out::println);  
  
    Consola.info("Total: " + banco.totalClientes() + " cliente(s).");  
}  
  
// Busca y muestra un cliente por su DNI  
public void buscarClientePorDni(String dni) {  
    banco.buscarClientePorDni(dni).ifPresentOrElse(  
        cliente -> {  
            System.out.println(cliente);  
            cliente.getCuentas().forEach(System.out::println);  
        },  
        () -> Consola.error("No se encontró cliente con DNI: " + dni));  
}  
  
// Abre una Caja de Ahorro para el cliente con el DNI que se pasa  
public void abrirCajaDeAhorro(String dni, double saldoInicial) {  
    Optional<Cliente> clienteOptional = banco.buscarClientePorDni(dni);  
  
    if (clienteOptional.isEmpty()) {  
        Consola.error("No existe cliente con DNI: " + dni);  
        return;  
    }  
  
    String numeroDeCuenta = banco.generarNumeroCuenta();  
    CajaDeAhorro cuenta = new CajaDeAhorro(numeroDeCuenta, saldoInicial,  
clienteOptional.get());  
    banco.registrarCuenta(cuenta);  
  
    Consola.exito("Caja de Ahorro abierta. Número: " + numeroDeCuenta);  
}  
  
// Abre una Cuenta Corriente para el cliente con el DNI que se pasa  
public void abrirCuentaCorriente(String dni, double saldoInicial, double  
limiteDescubierto) {  
    Optional<Cliente> clienteOptional = banco.buscarClientePorDni(dni);  
  
    if (clienteOptional.isEmpty()) {  
        Consola.error("No existe cliente con DNI: " + dni);  
        return;  
    }  
}
```

```
}

String numeroDeCuenta = banco.generarNumeroCuenta();
CuentaCorriente cuenta = new CuentaCorriente(numeroDeCuenta, saldoInicial,
    clienteOptional.get(), limiteDescubierto);
banco.registrarCuenta(cuenta);

Consola.exito("Cuenta Corriente abierta. Número: " + numeroDeCuenta
    + " | Descubierto: $" + limiteDescubierto);
}

// Muestra todas las cuentas de un cliente si es que este existe
public void verCuentasDeCliente(String dni) {
    banco.buscarClientePorDni(dni).ifPresentOrElse(
        cliente -> {
            if (cliente.getCuentas().isEmpty()) {
                Consola.info("El cliente no tiene cuentas.");
            } else {
                cliente.getCuentas().forEach(System.out::println);
            }
        },
        () -> Consola.error("No existe cliente con DNI: " + dni));
}

// Deposita un monto en una cuenta
public void depositar(String numeroDeCuenta, double monto) {
    banco.buscarCuenta(numeroDeCuenta).ifPresentOrElse(
        cuenta -> {
            if (cuenta.depositar(monto)) {
                Consola.exito(String.format("Depósito exitoso: $%.2f en
cuenta %s", monto, numeroDeCuenta));
                Consola.info("Nuevo saldo: $" + cuenta.getSaldo());
            } else {
                Consola.error("Monto inválido para depósito.");
            }
        },
        () -> Consola.error("No existe la cuenta: " + numeroDeCuenta));
}

// Extrae un monto de una cuenta
public void extraer(String numeroDeCuenta, double monto) {
    banco.buscarCuenta(numeroDeCuenta).ifPresentOrElse(
        cuenta -> {
            if (cuenta.extraer(monto)) {
                Consola.exito(String.format("Extracción exitosa: $%.2f de
cuenta %s", monto, numeroDeCuenta));
                Consola.info("Nuevo saldo: $" + cuenta.getSaldo());
            } else {
                Consola.error("Fondos insuficientes o monto inválido.");
            }
        },
        () -> Consola.error("No existe la cuenta: " + numeroDeCuenta));
}
```

```
        () -> Consola.error("No existe la cuenta: " + numeroDeCuenta));
    }

    public void transferir(String numeroDeOrigen, String numeroDeDestino, double
monto) {
        Optional<Cuenta> origenOptional = banco.buscarCuenta(numeroDeOrigen);
        Optional<Cuenta> destinoOptional = banco.buscarCuenta(numeroDeDestino);

        if (origenOptional.isEmpty()) {
            Consola.error("No existe la cuenta origen: " + numeroDeOrigen);
            return;
        }

        if (destinoOptional.isEmpty()) {
            Consola.error("No existe la cuenta destino: " + numeroDeDestino);
            return;
        }

        Cuenta origen = origenOptional.get();
        Cuenta destino = destinoOptional.get();

        if (!origen.extraer(monto)) {
            Consola.error("Fondos insuficientes en la cuenta origen.");
            return;
        }

        destino.depositar(monto);

        Consola.exito(String.format("Transferencia exitosa: $%.2f de %s a %s",
monto, numeroDeOrigen, numeroDeDestino));
    }

    // Muestra el historial completo de los movimientos de una cuenta
    public void verHistorial(String numeroDeCuenta) {
        banco.buscarCuenta(numeroDeCuenta).ifPresentOrElse(
            cuenta -> {
                Consola.subtitulo("Historial - Cuenta " + numeroDeCuenta);
                System.out.println(cuenta);

                if (cuenta.getHistorialMovimientos().isEmpty()) {
                    Consola.info("Sin movimientos registrados.");
                } else {
                    cuenta.getHistorialMovimientos().forEach(System.out::println);
                }
            },
            () -> Consola.error("No existe la cuenta: " + numeroDeCuenta));
    }

    public void verSaldo(String numeroDeCuenta) {
        banco.buscarCuenta(numeroDeCuenta).ifPresentOrElse(
```

```
        cuenta -> Consola.info(String.format("Saldo de cuenta %s: $%.2f",
numeroDeCuenta, cuenta.getSaldo()))),
        () -> Consola.error("No existe la cuenta: " + numeroDeCuenta));
    }

    // Muestra un resumen general del banco
    public void resumenGeneral() {
        Consola.titulo("RESUMEN GENERAL DEL BANCO");
        System.out.println(" " + banco);
        Consola.separador();

        double totalActivos = banco.getCuentas().stream()
            .mapToDouble(Cuenta::getSaldo)
            .sum();

        System.out.printf(" Total de activos en el sistema: $%.2f%n", totalActivos);
        System.out.printf(" Promedio de saldo por cuenta: $%.2f%n",
            banco.totalCuentas() > 0 ? totalActivos / banco.totalCuentas() : 0);
        Consola.separador();
    }
}
```

Clase : servicio/BancoService.java — logica de negocio con Optional y Stream

3.3. Capa de utilidades

3.3.1. Clase Consola

```
package utilidades;

import java.util.Scanner;

public class Consola {
    private static final String SEPARADOR = "=".repeat(70);
    private static final String COLOR_OK = "\u001B[32m"; // Verde
    private static final String COLOR_ERROR = "\u001B[31m"; // Rojo
    private static final String COLOR_INFO = "\u001B[36m"; // Cyan
    private static final String COLOR_RESET = "\u001B[0m";

    private Consola() {
    }

    public static void separador() {
        System.out.println(SEPARADOR);
    }

    public static void titulo(String texto) {
        System.out.println();
        separador();
        System.out.printf("%s%n", texto.toUpperCase());
    }
}
```

```
        separador();
    }

    public static void subtitulo(String texto) {
        System.out.println();
        System.out.println("\\textgreater\\textgreater{} " + texto);
        System.out.println("-".repeat(50));
    }

    public static void exito(String mensaje) {
        System.out.println(COLOR_OK + "[OK]" + mensaje + COLOR_RESET);
    }

    public static void error(String mensaje) {
        System.out.println(COLOR_ERROR + "[ERROR]" + mensaje + COLOR_RESET);
    }

    public static void info(String mensaje) {
        System.out.println(COLOR_INFO + "[INFO]" + mensaje + COLOR_RESET);
    }

    // Lee un número entero y valida el formato
    public static int leerEntero(Scanner scanner, String prompt) {
        while (true) {
            System.out.print(prompt + ": ");
            try {
                int valor = Integer.parseInt(scanner.nextLine().trim());
                return valor;
            } catch (NumberFormatException e) {
                error("Ingrese un número válido.");
            }
        }
    }

    // Lee un número decimal, valida el formato y que sea positivo
    public static double leerDouble(Scanner scanner, String prompt) {
        while (true) {
            System.out.print(prompt + ": $");
            try {
                double valor =
Double.parseDouble(scanner.nextLine().trim().replace(",", "."));
                if (valor < 0) {
                    error("El valor debe ser positivo.");
                } else {
                    return valor;
                }
            } catch (NumberFormatException e) {
                error("Ingrese un monto válido (ej: 1500.50).");
            }
        }
    }
}
```

```
}
```

Clase : utilidades/Consola.java — lectura robusta de entradas

3.4. Menú principal

3.4.1. Clase MenuPrincipal

```
package menu;

import java.util.Scanner;

import servicio.BancoService;
import utilidades.Consola;

public class MenuPrincipal {
    private final BancoService bancoService;
    private final Scanner scanner;

    public MenuPrincipal() {
        this.bancoService = new BancoService();
        this.scanner = new Scanner(System.in);
        cargarDatosDemo(); // Para poder hacer pruebas
    }

    // Inicia el ciclo principal del menú
    public void iniciar() {
        Consola.separador();
        System.out.println("BIENVENIDO AL SISTEMA BANCARIO");
        System.out.println("Desarrollado por: Facundo Luna | Programacion I ---
2026");
        Consola.separador();
        System.out.println();
        System.out.println("Este sistema permite gestionar clientes y cuentas
bancarias.");
        System.out.println("Desde el menú principal podrá: registrar clientes, abrir
cuentas,");
        System.out.println("realizar operaciones bancarias y consultar reportes e
historial.");
        System.out.println();
        Consola.separador();
        int opcion;

        do {
            mostrarMenuPrincipal();
            opcion = Consola.leerEntero(scanner, "Seleccione una opción");

            switch (opcion) {
                case 1 -> menuClientes();
                case 2 -> menuCuentas();
            }
        }
    }
}
```

```
        case 3 -> menuOperaciones();
        case 4 -> menuReportes();
        case 0 -> Consola.info("Cerrando sistema. Hasta pronto!");
        default -> Consola.error("Opción inválida. Intente nuevamente...");
    }
} while (opcion != 0);

scanner.close(); // Cerramos el Scanner para optimizar
}

private void mostrarMenuPrincipal() {
    Consola.separador();
    System.out.println("MENÚ PRINCIPAL");
    Consola.separador();
    System.out.println("1. Gestión de Clientes");
    System.out.println("2. Gestión de Cuentas");
    System.out.println("3. Operaciones Bancarias");
    System.out.println("4. Reportes e Historial");
    System.out.println("0. Salir");
    Consola.separador();
}

private void menuClientes() {
    int opcion;
    do {
        Consola.separador();
        System.out.println("GESTIÓN DE CLIENTES");
        Consola.separador();
        System.out.println("1. Registrar nuevo cliente");
        System.out.println("2. Listar todos los clientes");
        System.out.println("3. Buscar cliente por DNI");
        System.out.println("0. Volver al menú principal");
        Consola.separador();

        opcion = Consola.leerEntero(scanner, "Seleccione una opción");

        switch (opcion) {
            case 1 -> registrarCliente();
            case 2 -> bancoService.listarClientes();
            case 3 -> buscarCliente();
            case 0 -> Consola.info("Volviendo al menú principal...");
            default -> Consola.error("Opción inválida.");
        }
    } while (opcion != 0);
}

private void menuCuentas() {
    int opcion;
    do {
        Consola.separador();
        System.out.println("GESTIÓN DE CUENTAS");
```



```
        Consola.separador();
        System.out.println("1. Abrir Caja de Ahorro");
        System.out.println("2. Abrir Cuenta Corriente");
        System.out.println("3. Ver cuentas de un cliente");
        System.out.println("0. Volver al menú principal");
        Consola.separador();

        opcion = Consola.leerEntero(scanner, "Seleccione una opción");

        switch (opcion) {
            case 1 -> abrirCajaDeAhorro();
            case 2 -> abrirCuentaCorriente();
            case 3 -> verCuentasCliente();
            case 0 -> Consola.info("Volviendo al menú principal...");
            default -> Consola.error("Opción inválida.");
        }
    } while (opcion != 0);
}

private void menuOperaciones() {
    int opcion;
    do {
        Consola.separador();
        System.out.println("OPERACIONES BANCARIAS");
        Consola.separador();
        System.out.println("1. Depositar");
        System.out.println("2. Extraer");
        System.out.println("3. Transferir entre cuentas");
        System.out.println("0. Volver al menú principal");
        Consola.separador();

        opcion = Consola.leerEntero(scanner, "Seleccione una opción");

        switch (opcion) {
            case 1 -> depositar();
            case 2 -> extraer();
            case 3 -> transferir();
            case 0 -> Consola.info("Volviendo al menú principal...");
            default -> Consola.error("Opción inválida.");
        }
    } while (opcion != 0);
}

private void menuReportes() {
    int opcion;
    do {
        Consola.separador();
        System.out.println("REPORTES E HISTORIAL");
        Consola.separador();
        System.out.println("1. Ver historial de una cuenta");
        System.out.println("2. Ver saldo de una cuenta");
```

```
        System.out.println("3. Resumen general del banco");
        System.out.println("0. Volver al menú principal");
        Consola.separador();

        opcion = Consola.leerEntero(scanner, "Seleccione una opción");

        switch (opcion) {
            case 1 -> verHistorial();
            case 2 -> verSaldo();
            case 3 -> bancoService.resumenGeneral();
            case 0 -> Consola.info("Volviendo al menú principal...");
            default -> Consola.error("Opción inválida.");
        }
    } while (opcion != 0);
}

private void registrarCliente() {
    Consola.subtitulo("Registrar Nuevo Cliente");
    System.out.print("Nombre: ");
    String nombre = scanner.nextLine();
    System.out.print("Apellido: ");
    String apellido = scanner.nextLine();
    System.out.print("DNI: ");
    String dni = scanner.nextLine();
    System.out.print("Email: ");
    String email = scanner.nextLine();

    bancoService.registrarCliente(nombre, apellido, dni, email);
}

private void buscarCliente() {
    Consola.subtitulo("Buscar Cliente por DNI");
    System.out.print("Ingrese DNI: ");
    String dni = scanner.nextLine();
    bancoService.buscarClientePorDni(dni);
}

private void abrirCajaDeAhorro() {
    Consola.subtitulo("Abrir Caja de Ahorro");
    System.out.print("DNI del cliente: ");
    String dni = scanner.nextLine();
    double saldoInicial = Consola.leerDouble(scanner, "Saldo inicial");
    bancoService.abrirCajaDeAhorro(dni, saldoInicial);
}

private void abrirCuentaCorriente() {
    Consola.subtitulo("Abrir Cuenta Corriente");
    System.out.print("DNI del cliente: ");
    String dni = scanner.nextLine();
    double saldoInicial = Consola.leerDouble(scanner, "Saldo inicial");
}
```

```
        double limiteDescubierto = Consola.leerDouble(scanner, "Límite de
descubierto");
        bancoService.abrirCuentaCorriente(dni, saldoInicial, limiteDescubierto);
    }

    private void verCuentasCliente() {
        Consola.subtitulo("Cuentas del Cliente");
        System.out.print("DNI del cliente: ");
        String dni = scanner.nextLine();
        bancoService.verCuentasDeCliente(dni);
    }

    private void depositar() {
        Consola.subtitulo("Depósito");
        System.out.print("Número de cuenta: ");
        String nroCuenta = scanner.nextLine();
        double monto = Consola.leerDouble(scanner, "Monto a depositar");
        bancoService.depositar(nroCuenta, monto);
    }

    private void extraer() {
        Consola.subtitulo("Extracción");
        System.out.print("Número de cuenta: ");
        String nroCuenta = scanner.nextLine();
        double monto = Consola.leerDouble(scanner, "Monto a extraer");
        bancoService.extraer(nroCuenta, monto);
    }

    private void transferir() {
        Consola.subtitulo("Transferencia");
        System.out.print("Cuenta origen : ");
        String origen = scanner.nextLine();
        System.out.print("Cuenta destino: ");
        String destino = scanner.nextLine();
        double monto = Consola.leerDouble(scanner, "Monto a transferir");
        bancoService.transferir(origen, destino, monto);
    }

    private void verHistorial() {
        Consola.subtitulo("Historial de Movimientos");
        System.out.print("Número de cuenta: ");
        String nroCuenta = scanner.nextLine();
        bancoService.verHistorial(nroCuenta);
    }

    private void verSaldo() {
        Consola.subtitulo("Consulta de Saldo");
        System.out.print("Número de cuenta: ");
        String nroCuenta = scanner.nextLine();
        bancoService.verSaldo(nroCuenta);
    }
}
```

```
private void cargarDatosDemo() {  
    bancoService.registrarCliente("Juan", "García", "12345678", "juan@email.com");  
    bancoService.registrarCliente("María", "López", "87654321",  
    "maria@email.com");  
    bancoService.abrirCajaDeAhorro("12345678", 5000.0);  
    bancoService.abrirCuentaCorriente("87654321", 2000.0, 10000.0);  
    Consola.info("Datos de prueba cargados correctamente.");  
}  
}
```

Clase : menu/MenuPrincipal.java — interfaz de usuario por consola

3.4.2. Clase App — punto de entrada

```
import menu.MenuPrincipal;  
  
public class App {  
    public static void main(String[] args) throws Exception {  
        MenuPrincipal menu = new MenuPrincipal();  
        menu.iniciar();  
    }  
}
```

Clase : App.java — arranque de la aplicacion

4. Capturas de ejecución

A continuación se presentan ejemplos reales de ejecución del sistema, organizados por funcionalidad. Los datos de prueba iniciales (*García, Juan* y *López, María*) son cargados automáticamente al iniciar la aplicación.

4.1. Inicio del sistema y datos de prueba

Al arrancar, el sistema carga clientes y cuentas de ejemplo y presenta el menú principal con las cuatro categorías de operaciones disponibles.

```
[OK] Cliente registrado: García, Juan
[OK] Cliente registrado: López, María
[OK] Caja de Ahorro abierta. Número: CBU-1001
[OK] Cuenta Corriente abierta. Número: CBU-1002 | Descubierto: $10000.0
[INFO] Datos de prueba cargados correctamente.

=====
BIENVENIDO AL SISTEMA BANCARIO
Desarrollado por: Facundo Luna | Programacion I -- 2026
=====

Este sistema permite gestionar clientes y cuentas bancarias.
Desde el menú principal podrá: registrar clientes, abrir cuentas,
realizar operaciones bancarias y consultar reportes e historial.

=====
MENÚ PRINCIPAL
=====
1. Gestión de Clientes
2. Gestión de Cuentas
3. Operaciones Bancarias
4. Reportes e Historial
0. Salir
=====
Seleccione una opción:
```

4.2. Gestión de clientes

Se registra un nuevo cliente (*Uzumaki, Naruto*), se lista el padrón completo y se lo busca por DNI para verificar el alta.

```
Seleccione una opción: 1

» Registrar Nuevo Cliente
-----
Nombre: Naruto
Apellido: Uzumaki
DNI: 38735411
Email: naruto@gmail.com
[OK] Cliente registrado: Uzumaki, Naruto

Seleccione una opción: 2

» Clientes Registrados
-----
Cliente: García, Juan      | DNI: 12345678 | Email: juan@email.com |
Cuentas: 1
Cliente: López, María     | DNI: 87654321 | Email: maria@email.com |
Cuentas: 1
Cliente: Uzumaki, Naruto  | DNI: 38735411 | Email: naruto@gmail.com|
Cuentas: 0
[INFO] Total: 3 cliente(s).

Seleccione una opción: 3

» Buscar Cliente por DNI
-----
Ingrese DNI: 38735411
Cliente: Uzumaki, Naruto | DNI: 38735411 | Email: naruto@gmail.com |
Cuentas: 0
```

4.3. Apertura de cuenta y consulta

Se abre una Caja de Ahorro para el cliente recién registrado y se consultan sus cuentas para confirmar la apertura y el saldo inicial.

```
Seleccione una opción: 1  (Abrir Caja de Ahorro)

» Abrir Caja de Ahorro
-----
DNI del cliente: 38735411
Saldo inicial: $10000
[OK] Caja de Ahorro abierta. Número: CBU-1003

Seleccione una opción: 3  (Ver cuentas de un cliente)

» Cuentas del Cliente
-----
DNI del cliente: 38735411
Cuenta CBU-1003 | Tipo: Caja de Ahorro | Titular: Uzumaki, Naruto
| Saldo: $10000,00 | Tasa: 5,0%
```

4.4. Transferencia entre cuentas

Se transfieren \$5000 desde la cuenta de Naruto (CBU-1003) hacia la cuenta de García (CBU-1001). El sistema valida el saldo disponible y confirma la operación.

```
Seleccione una opción: 3  (Transferir entre cuentas)

» Transferencia
-----
Cuenta origen   : CBU-1003
Cuenta destino  : CBU-1001
Monto a transferir: $5000
[OK] Transferencia exitosa: $5000,00 de CBU-1003 a CBU-1001
```

4.5. Historial de movimientos y saldo

Se consulta el historial de CBU-1003. Se observan los dos movimientos registrados: el depósito inicial por apertura y la extracción correspondiente a la transferencia saliente.

```
Seleccione una opción: 1  (Ver historial de una cuenta)

» Historial de Movimientos
-----
Número de cuenta: CBU-1003

» Historial -- Cuenta CBU-1003
-----
Cuenta CBU-1003 | Tipo: Caja de Ahorro | Titular: Uzumaki, Naruto
| Saldo: $5000,00 | Tasa: 5,0%

[09/06/2026 23:05:44] Depósito   | Monto: $ 10000,00 | Saldo: $ 10000,00
| Apertura de cuenta
[09/06/2026 23:07:16] Extracción | Monto: $  5000,00 | Saldo: $  5000,00
| Extracción en efectivo

[INFO] Volviendo al menú principal...

Seleccione una opción: 0
[INFO] Cerrando sistema. Hasta pronto!
```

Los ejemplos verifican de forma integral: carga inicial de datos, registro y búsqueda de clientes, apertura de cuentas, transferencia con validación de saldo, y consulta de historial con marca de tiempo por cada movimiento.